

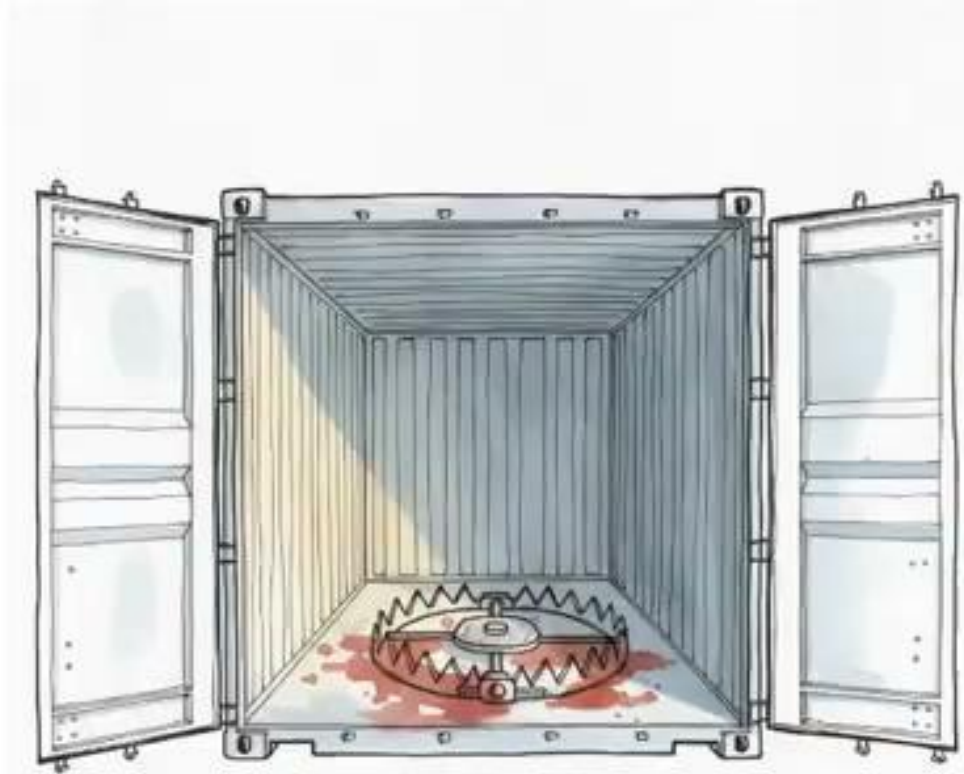
Modernizing SQL Server in Kubernetes

Alfred Bach

Principal Instructor



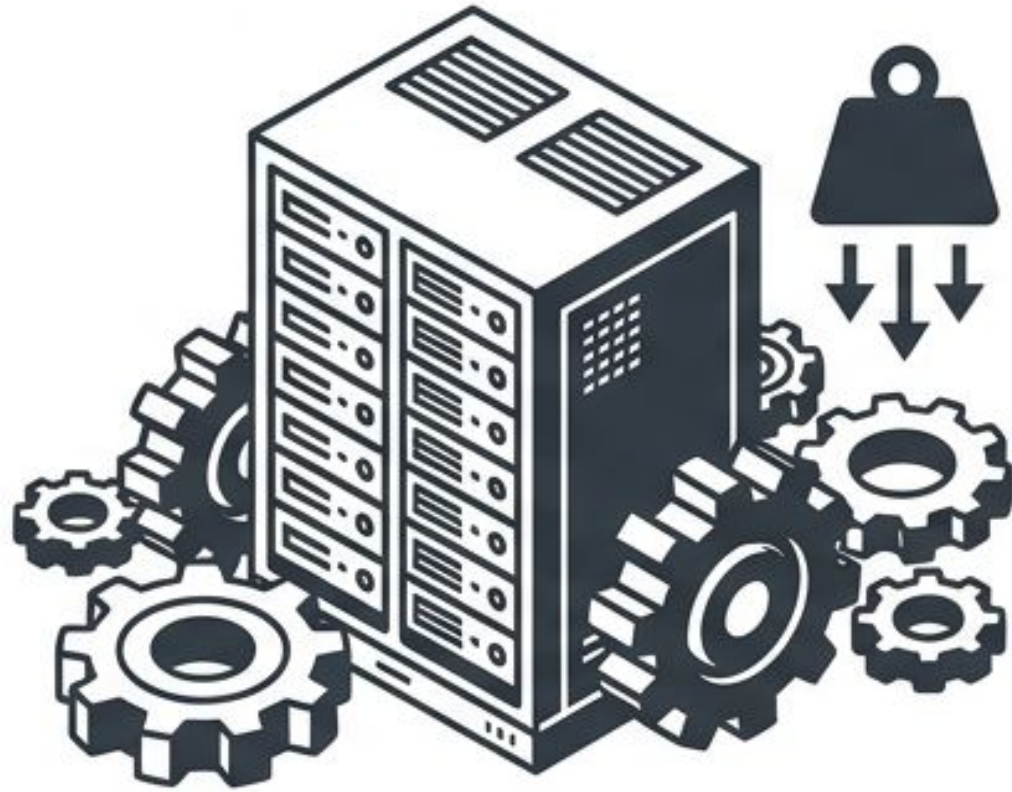
SQL in Containers



The Monolithic Database Bottleneck

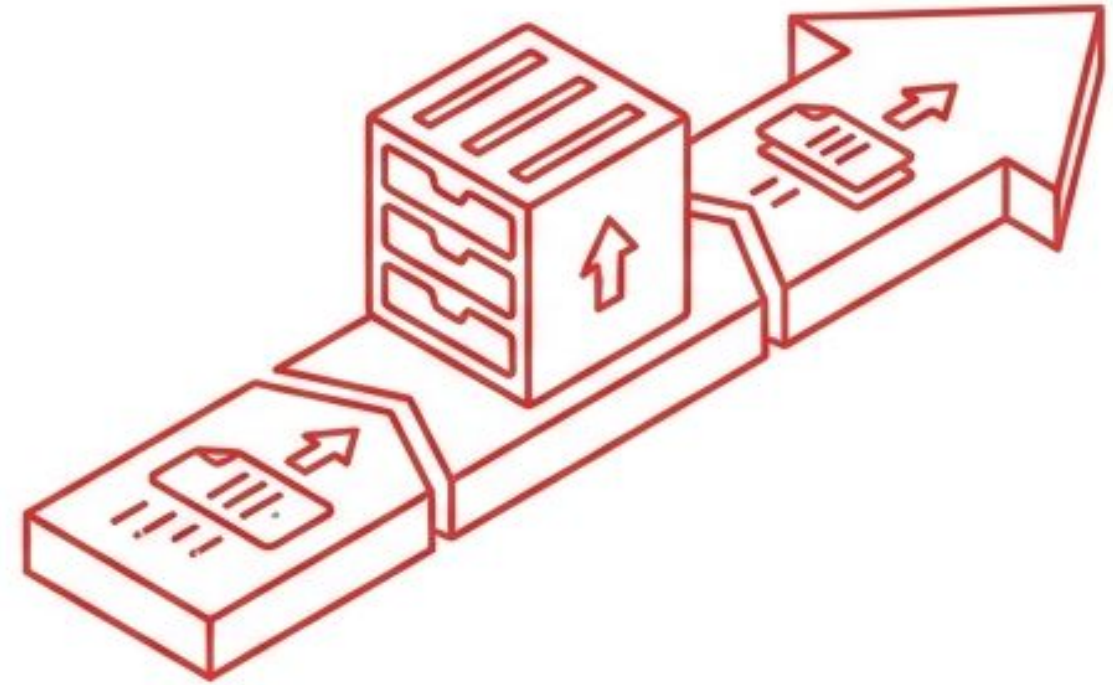
Legacy database management breaks the continuous delivery pipeline.

The Legacy Reality



- Manual provisioning sequences taking days or weeks.
- Heavyweight VMs tightly coupled to specific host hardware.

The Containerized Catalyst



- Infrastructure as Code (IaC) enabling deployment in seconds.
- Environment consistency across development, testing, and production.
- Built-in workload portability.

Enterprise-Grade Container Ecosystems

Modern platforms handle complex workloads with built-in security and orchestration guarantees.



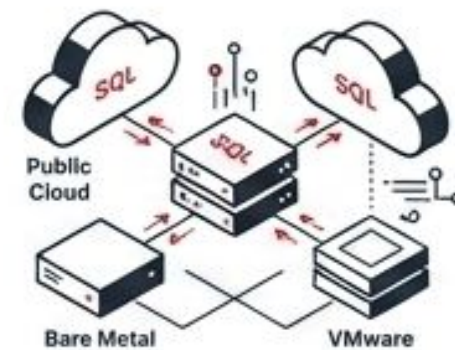
Orchestration & Automation

Kubernetes handles scaling, desired-state maintenance, and automated failover.



Enhanced Security Isolation

Running applications as non-root users by default, ensuring strict process isolation without requiring privileged containers.

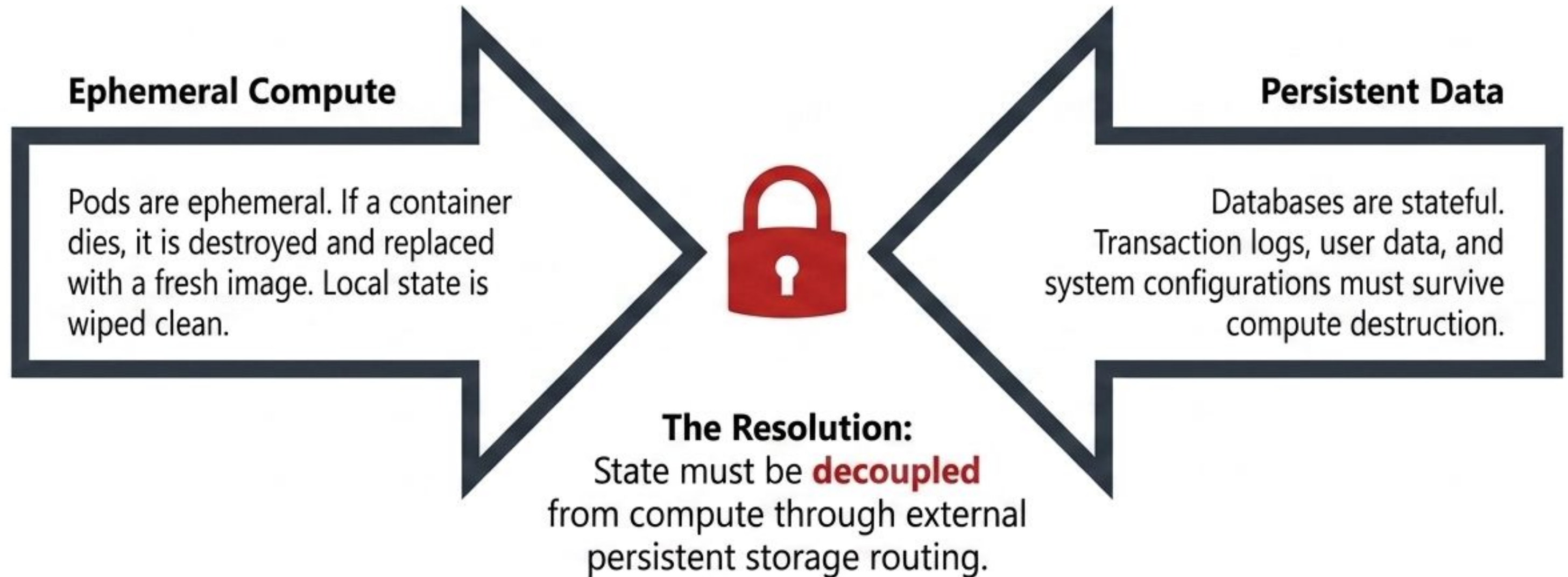


Infrastructure Agnostic

Consistent behavior across bare metal, VMware, or public cloud providers.

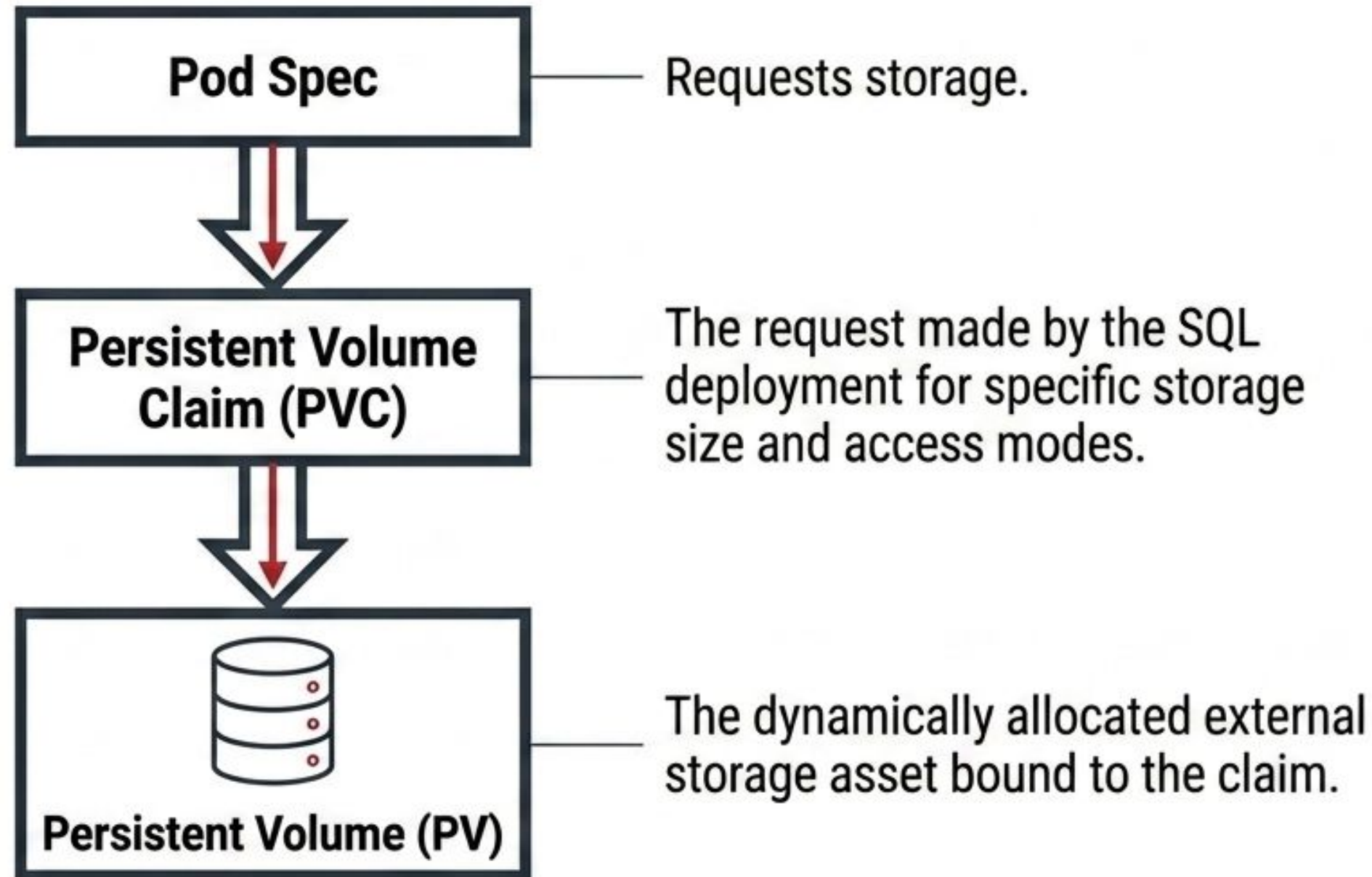
The Stateful Paradox

Kubernetes natively assumes ephemerality. SQL Server demands permanence.



Anchoring State with Persistent Volume Claims

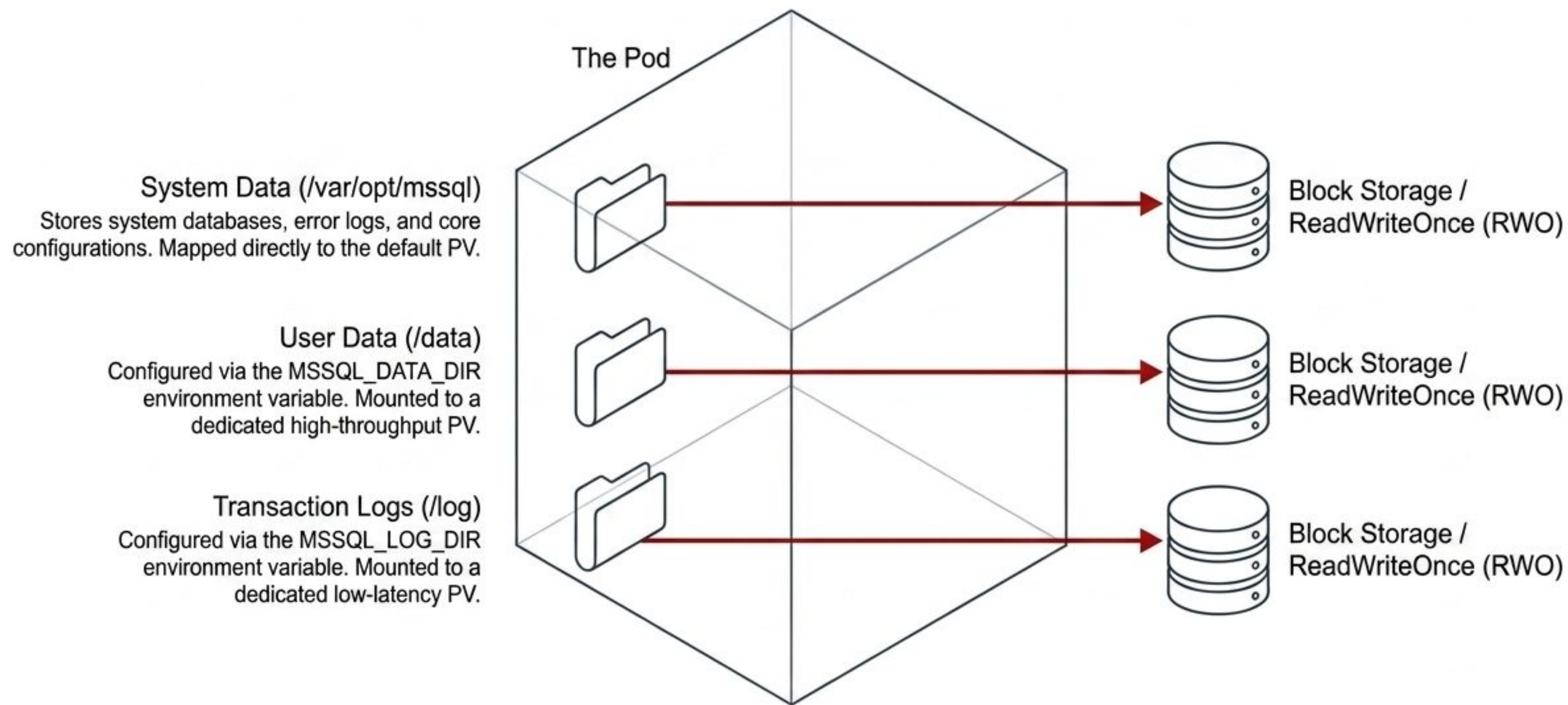
Dynamic storage provisioning bridges the gap between ephemeral pods and physical disks.



Storage Class:
Defines the storage profile (e.g., Azure managed-premium).

Anatomy of a Stateful SQL Pod

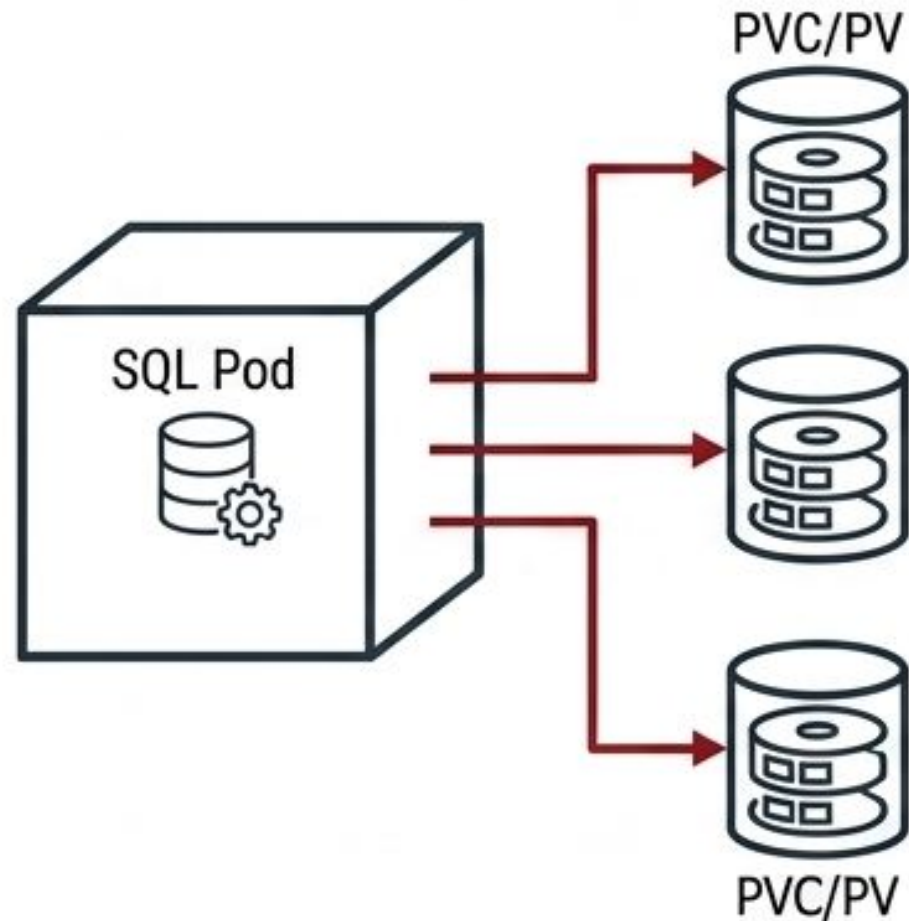
Optimal storage design requires isolating I/O profiles using dedicated Block/RWO storage.



Storage Survivability in Action

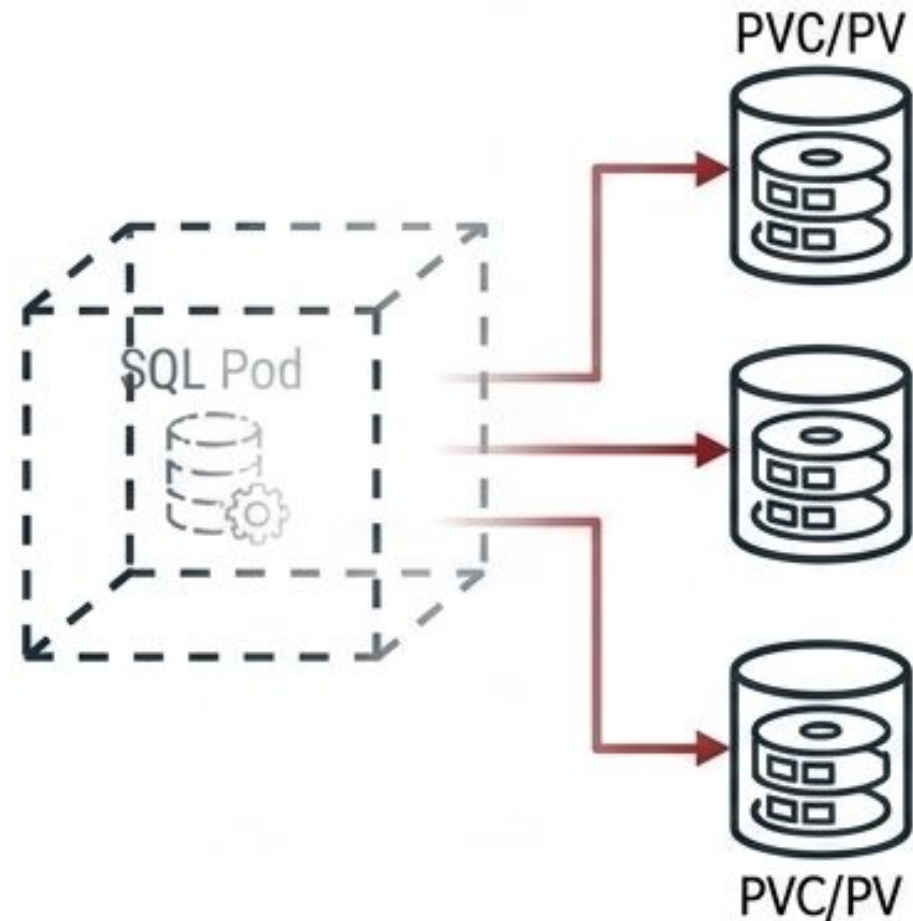
When a Pod dies, the independent PVCs remain anchored and safe.

Phase 1: Normal Operation



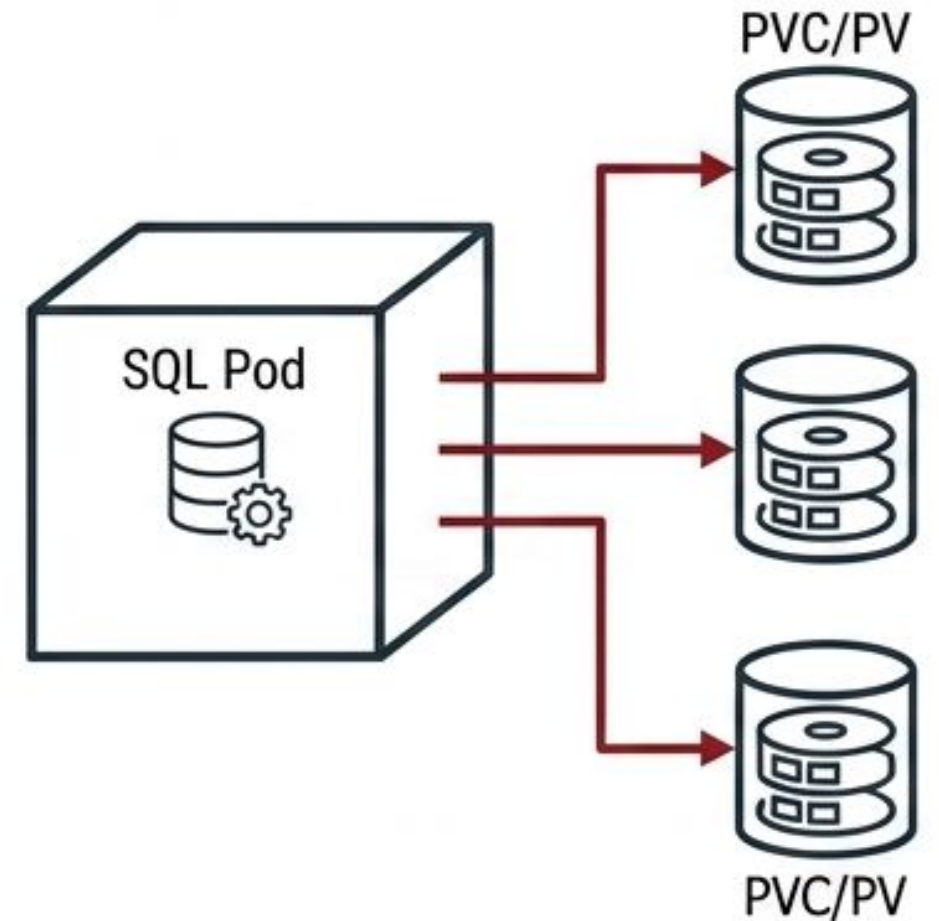
SQL Pod is running. Data is actively written to the three external PVCs.

Phase 2: Pod Failure



The compute unit is destroyed, but the Persistent Volumes transition to a holding state, retaining all data.

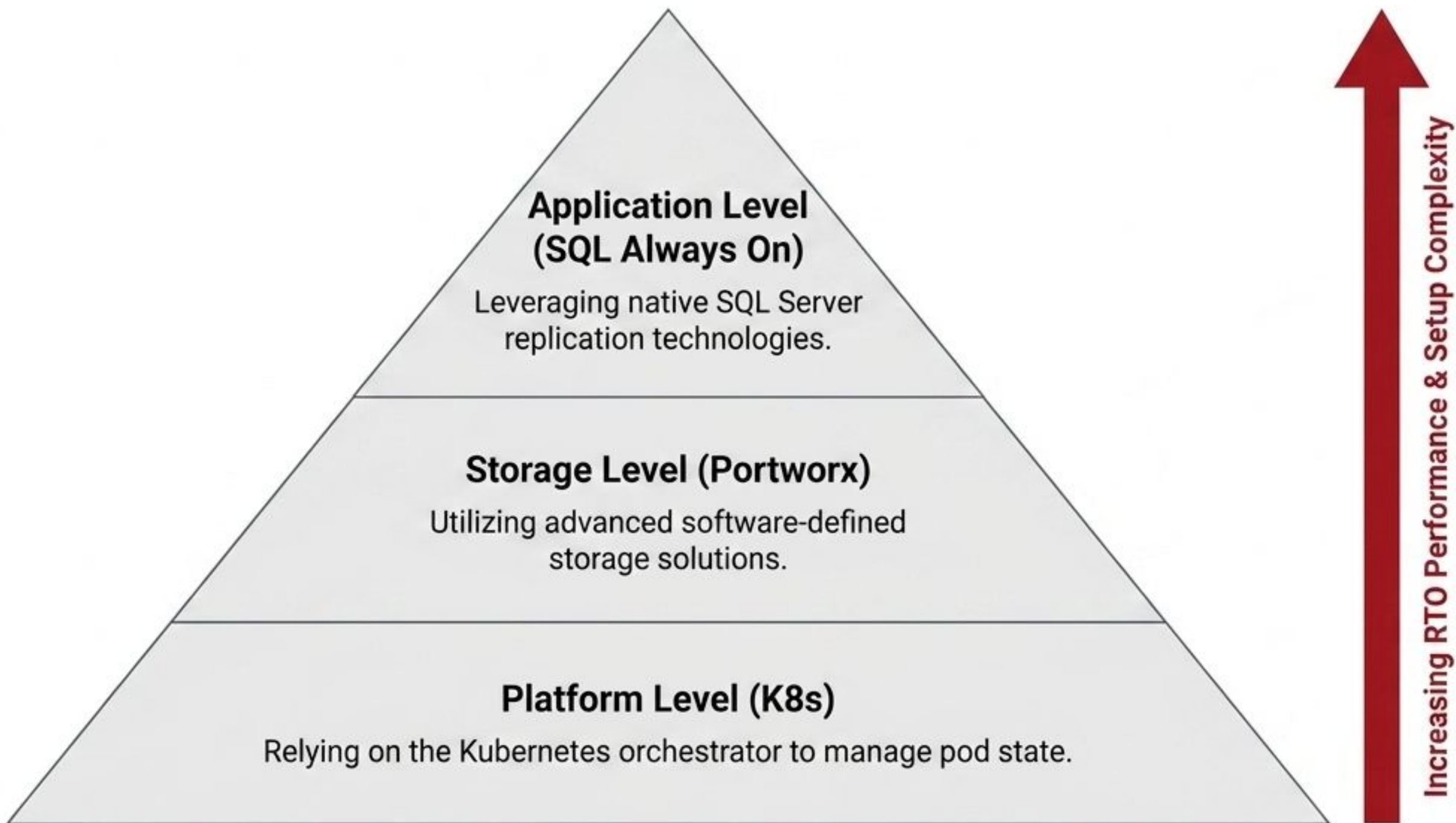
Phase 3: Rescheduling



Kubernetes spins up a new SQL container. It attaches to the exact same PVCs. SQL Server reads the system databases, finds the user databases, and resumes normal operation.

Layering Database Resilience

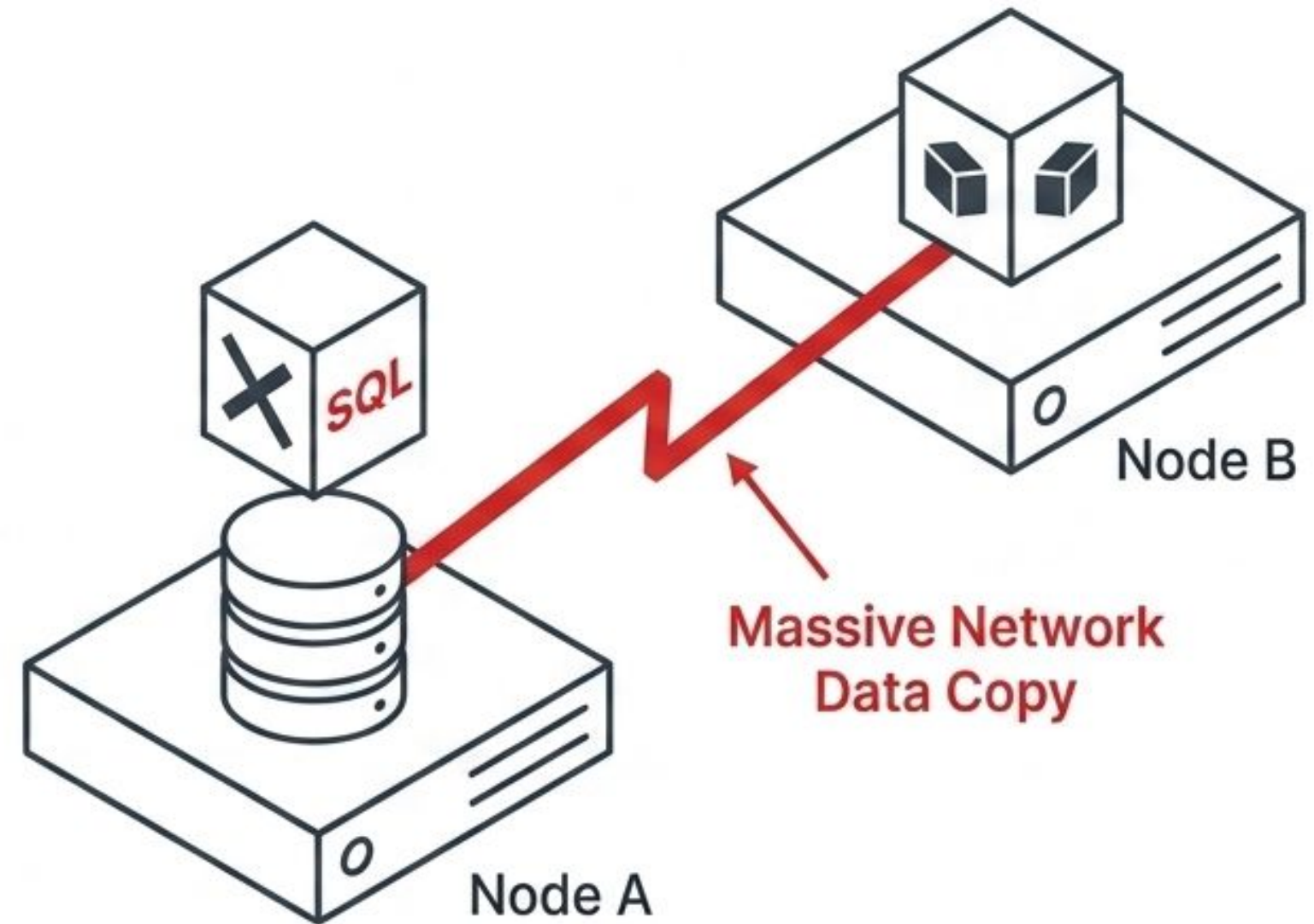
High Availability for SQL in containers requires choosing the right operational layer.



Strategy 1: Native Kubernetes Rescheduling

Automated recovery, but crippled by data gravity.

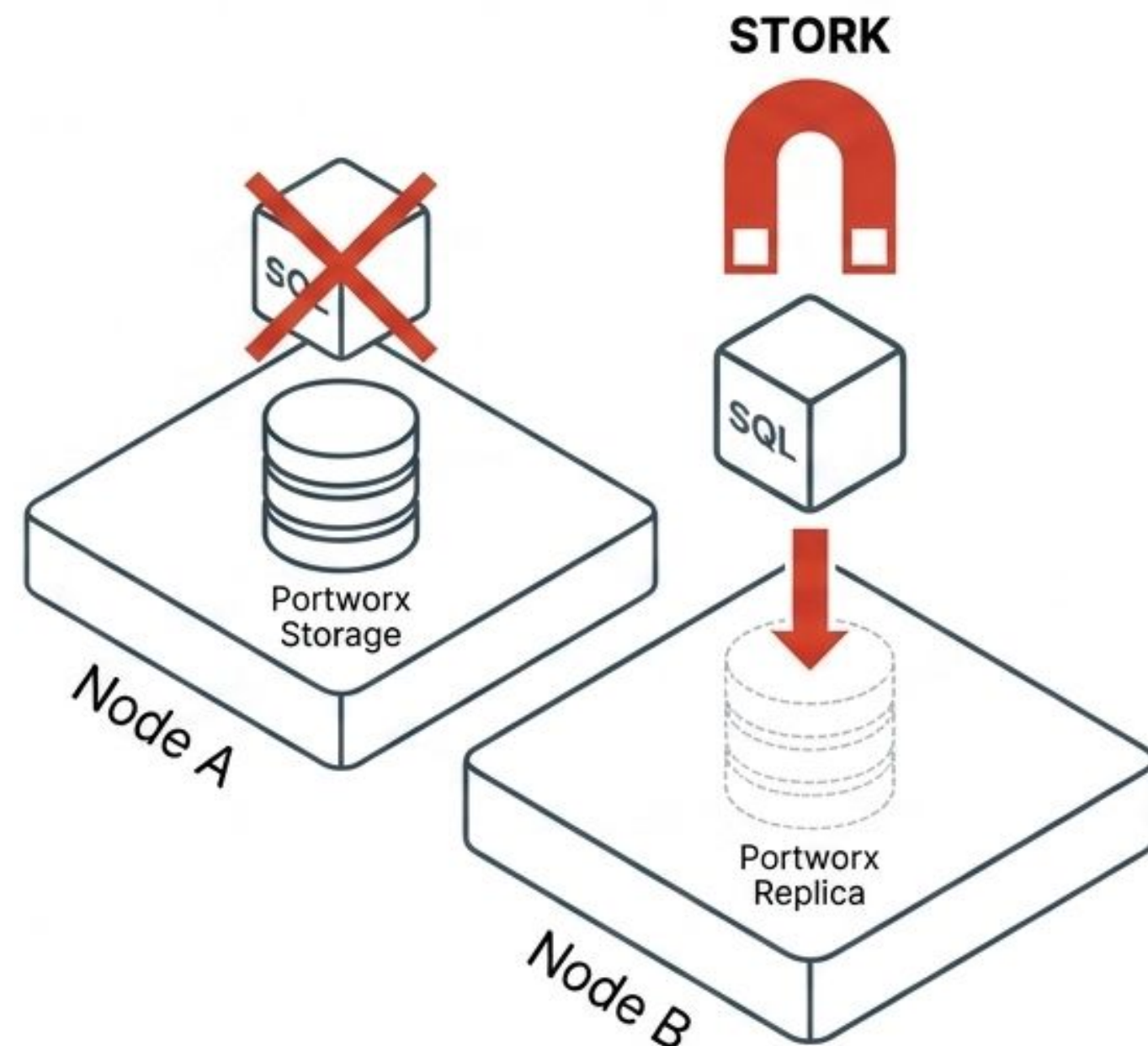
- **The Mechanism:** If a node fails, Kubernetes automatically schedules a new SQL pod on a healthy node.
- **The Flaw:** Standard Kubernetes schedulers are not data-aware. The new pod may be placed on a node far away from where the persistent data physically resides.
- **The Result:** Heavy network copying, latency spikes, and extended Recovery Time Objectives (RTO).



Strategy 2: Portworx with STORK

Data-aware scheduling ensuring zero RPO and rapid compute colocation.

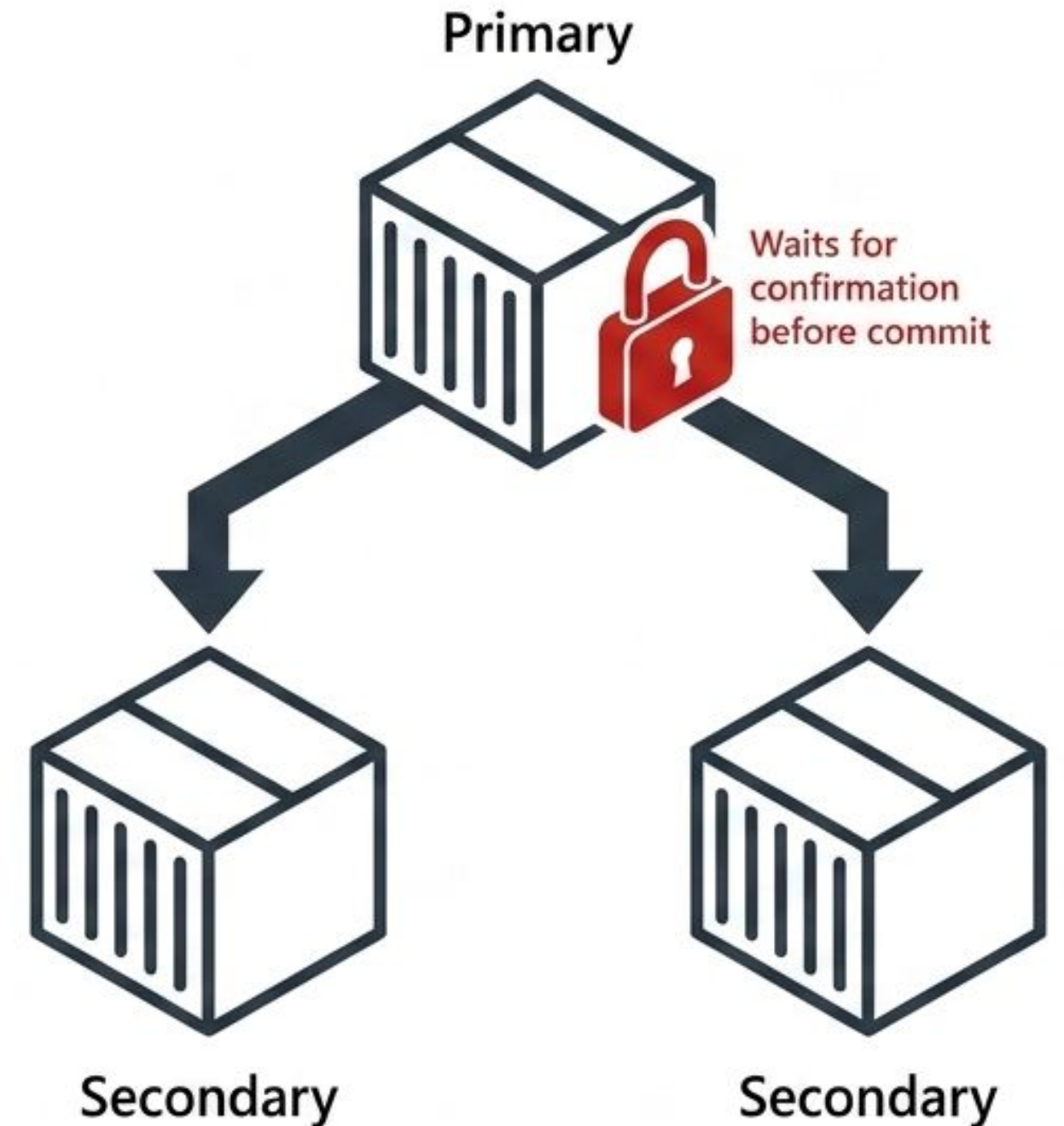
- **Synchronous Replication:** Portworx replicates the backing volumes across nodes, ensuring zero Recovery Point Objective (RPO) within a metropolitan area (max 15ms latency).
- **STORK (Storage Orchestrator for Kubernetes):** Acts as a placement magnet. When a pod dies, STORK intercepts the scheduler and pulls the new SQL pod directly to the node where the Portworx storage replica already physically resides.
- **The Result:** Near-zero RTO. No massive network data copies required.



Strategy 3: SQL Server Always On Availability Groups

Application-level orchestration for mission-critical transactional consistency

- **The Mechanism:** Relies on Pacemaker for cluster orchestration and active secondary replicas.
- **Data Integrity:** Secondary replicas must log the transaction data before the primary replica commits the transaction (REQUIRED_SYNCHRONIZED_SECONDARIES_TO_COMMIT).
- **State Tracking:** Pacemaker tracks replica health via sequence numbers updated in sys.availability_groups.
- **The Result:** Maximum data protection and seamless failover, but requires the highest configuration overhead and specific licensing tiers.



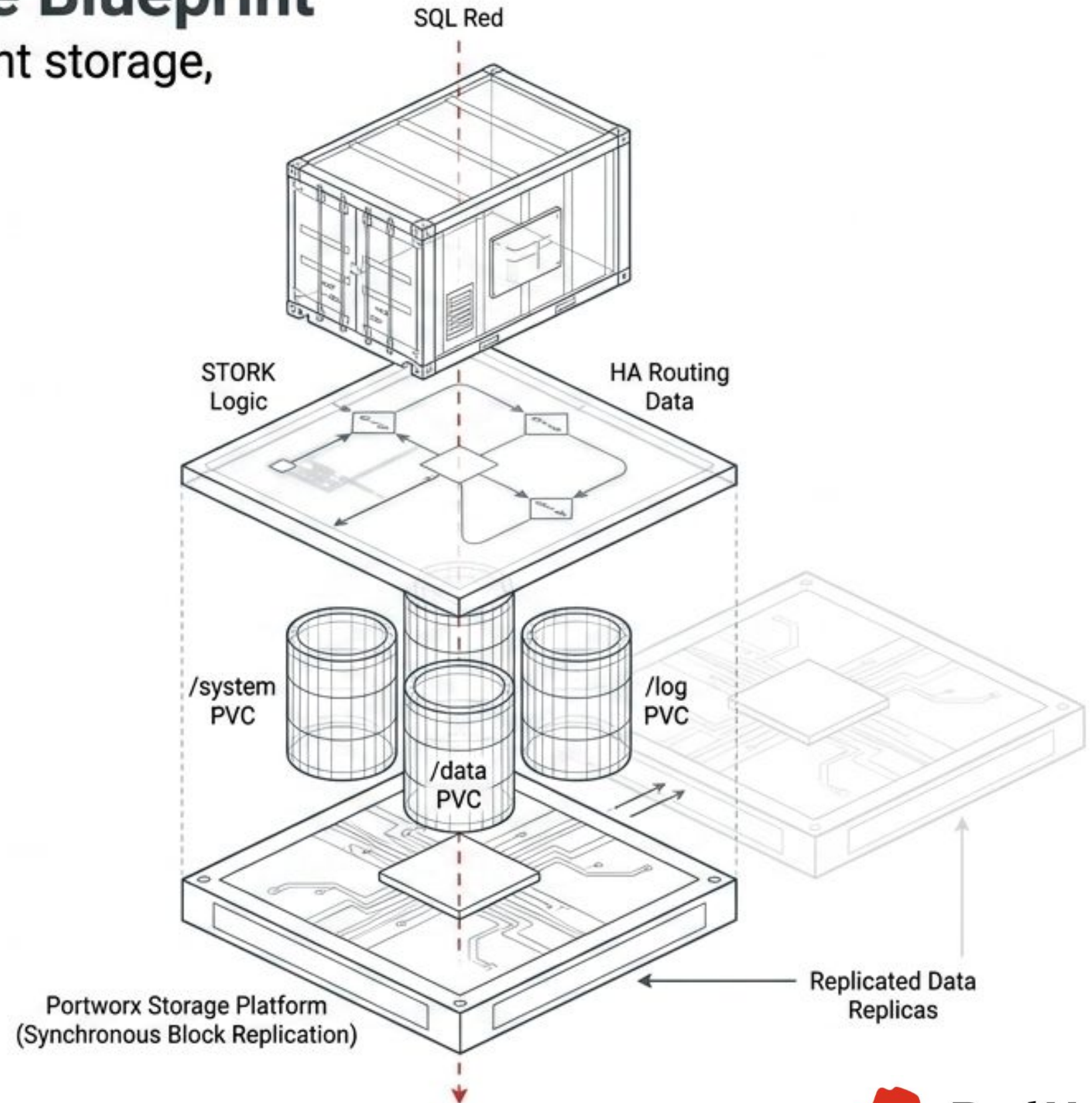
High Availability Diagnostic Matrix

Strategy	Operating Layer	Setup Complexity	Data Awareness	RTO (Recovery Time)
Native K8s Rescheduling	Platform	Low	None	Minutes (Network Dependent)
Portworx + STORK	Storage	Medium	High (Magnetic Placement)	Seconds (Local Attach)
Always On Availability Groups	Application	High (Pacemaker)	Absolute (Tx Log Sync)	Near-Zero

The Unified Enterprise Architecture Blueprint

Combining resilient orchestration, routed persistent storage, and data-aware scheduling

- **Compute:** Lightweight, non-root SQL Server Container.
- **Orchestration:** Kubernetes Deployment managing pod lifecycle.
- **HA Routing:** STORK ensuring compute is matched to local data replicas.
- **Persistence:** 3-Tier PVC routing (/system, /data, /log) mapped to Block/RWO.
- **Replication:** Portworx providing synchronous block-level replication across nodes.



The Business Reality Check: Financial Toxicity

Technical success does not guarantee business viability.



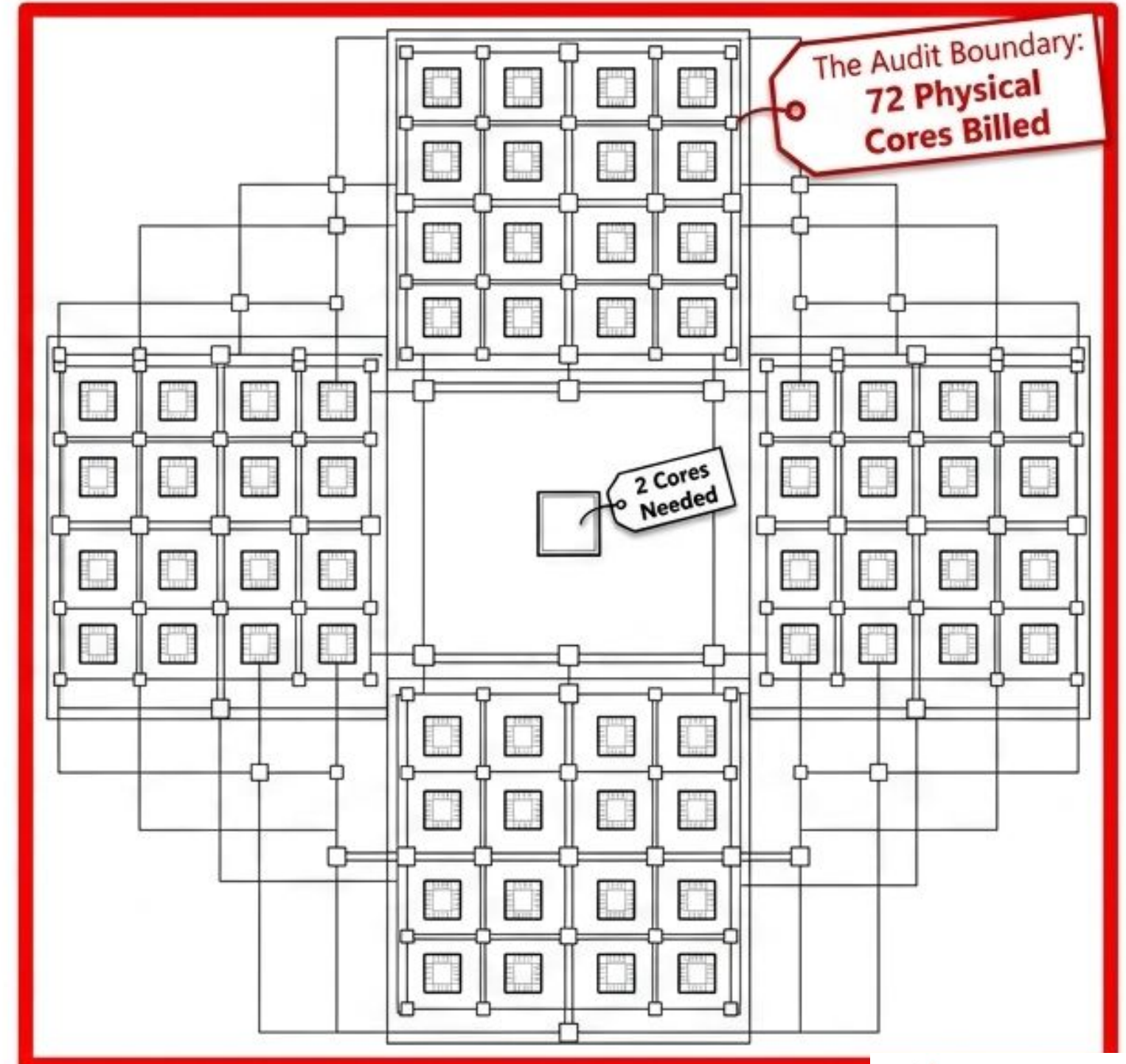
"The key problem with Microsoft database licensing is that they look at physical cores. It doesn't matter how many cores or fractional cores your application actually consumes in a container."

- If your software license costs exceed 3x your infrastructure bill, technical modernization has created financial toxicity.
- Auditors are aggressively targeting containerized SQL deployments for compliance breaches.

The Licensing Trap: Core-Based Auditing

Containers do not shield you from underlying hardware exposure.

- **The Technical Reality:** You configure your SQL Deployment to request exactly 2 CPU cores.
- **The Licensing Reality:** Your Kubernetes cluster consists of 3 physical hosts, each with 24 cores.
- **The Audit Math:** Because the container *could* theoretically be scheduled on any node, Microsoft requires you to license the entire cluster boundary. You must pay for 72 cores.



Strategies for Licensing Mitigation

How to protect the modernization budget without sacrificing architecture.



1. Infrastructure Isolation

Use Kubernetes node labels, taints, and tolerations to restrict SQL pods to specific, highly constrained physical nodes. License only those nodes.



2. Shift to Managed DBaaS

Offload the licensing complexity entirely by migrating to Azure SQL Managed Instance or Amazon RDS, consuming the database as a service rather than hosting it in-cluster.



3. The Open-Source Pivot

Re-platform. Migrate schemas from SQL Server to PostgreSQL or MySQL, eliminating commercial core-licensing restrictions entirely while maintaining container agility.



Modernization Requires Holistic Architecture

Running SQL Server in containers is technically highly effective, provided you:

1. Isolate I/O paths via dedicated Block/RWO PVCs.
2. Ensure data-aware High Availability to prevent network saturation.
3. Architect your physical nodes explicitly to contain licensing blast radiuses.

Engineering brilliant systems is only half the job. The other half is ensuring the business can afford to run them.

 linkedin.com/company/red-hat

 youtube.com/@redhat

 facebook.com/RedHat

 x.com/RedHat